

Teórica 7.4

"Database System Concepts", A. Silberschatz, H. Korth, S. Sudarshan

Procesamiento de consultas: Capítulo 15

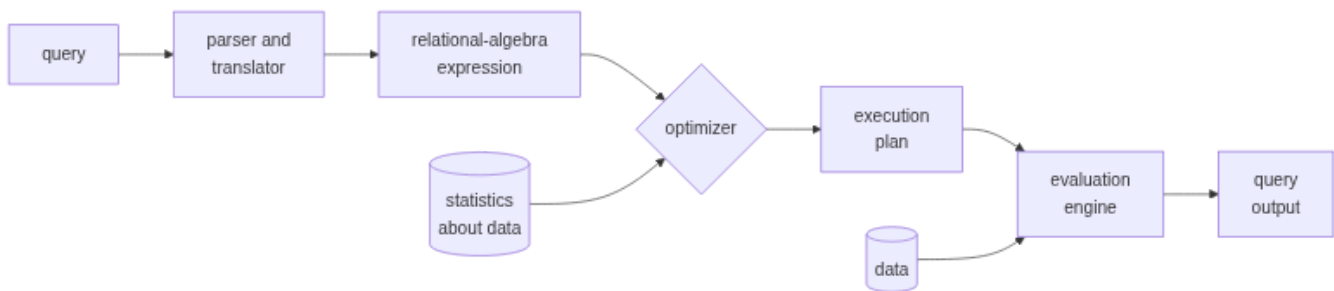
Chapter 15 - Query Processing

Query processing → the activities involved in extracting data from a database. The steps include *translating queries expressed in a high-level database language (SQL) into expressions that can be used at the physical level of the file system*, a variety of *query-optimization transformations*, and the *actual evaluation of queries*.

15.1 - Overview of Query Processing

The three basic steps are:

1. **Parsing and translation.**
2. **Optimization.**
3. **Evaluation.**



Before query processing can begin, the system must *translate the query into a usable form*. A language such as SQL is suitable for human use, but is *ill-suited to be the system's internal representation of a query*. A more useful internal representation is one based on the *extended relational algebra*.

The **parser** checks the syntax and verifies that the relation names appearing in the query are names of relations in the database. The query is then *translated into a relational-algebra expression*.

Given a query, there are generally *various methods for computing the answer*. For example, the join can be computed with a nested loop, sort-merge, or hash strategy.

Furthermore, for any given query, there can be *many equivalent relational-algebra expressions*.

For example, a selection $\sigma_{salary < 75000}(\Pi_{salary}(instructor))$ and the expression $\Pi_{salary}(\sigma_{salary < 75000}(instructor))$ produce the *same result*, but the second one can *potentially save a significant amount of effort* because the selection is done first.

Each relational-algebra operation can be evaluated using one of several *different algorithms*. An **evaluation plan** (also called a **query-execution plan** or **query plan**) defines exactly what algorithm to use for each operation and how the execution of the operations should be coordinated.

The query optimizer tries to find the *cheapest evaluation plan* for a query, or at least one that is *not much more expensive than the cheapest one*.

The cost of query evaluation is measured in terms of a number of different resources, including disk accesses, CPU time, and even the cost of communication. In *large database systems, the cost to access data from disk is usually the most important cost*, since disk accesses are slow compared with in-memory operations.

We use the number of **block transfers from disk** and the number of **disk seeks** as the cost measures.

If disk data is already in a **buffer**, we do *not need to account for the cost* of reading that data from disk. However, it is hard to take the effects of buffering into account, so we typically assume **worst case**: that data will have to be read from disk.

$t_T \rightarrow$ time to **transfer one block**.

$t_S \rightarrow$ time for one **seek**.

Cost for b block transfers plus S seeks: $b \cdot t_T + S \cdot t_S$.

We do **not** include the cost of writing the final output of the query to disk.

15.3 - Selection Operation

15.3.1 - Selections Using File Scans and Indices

A1 — Linear search (full scan). The system scans each file block and tests all records to see whether they satisfy the selection condition.

Cost estimate: $t_S + b_r \cdot t_T$

Where b_r denotes the number of blocks in the file. A single initial seek is required, and then a sequential scan of every block.

If the selection is on a *key attribute with an equality condition*, the system can stop when it finds the required record. In this case, *the average cost is reduced to half of the blocks*:

$$t_S + (b_r/2) \cdot t_T$$

Linear search can be applied to any file, *regardless of the ordering of the file, the availability of indices, or the nature of the selection operation*. It may be the only option when the condition involves an attribute that is not indexed.

A2 — Primary index (clustering index), equality on key.

Cost: $(h_i + 1) \cdot (t_S + t_T)$

Where h_i is the height of the index tree.

We traverse the tree from root to leaf (h_i levels) + 1 more access to fetch the actual record from the file. Each level is a *separate block on disk*, requiring one seek + one transfer.

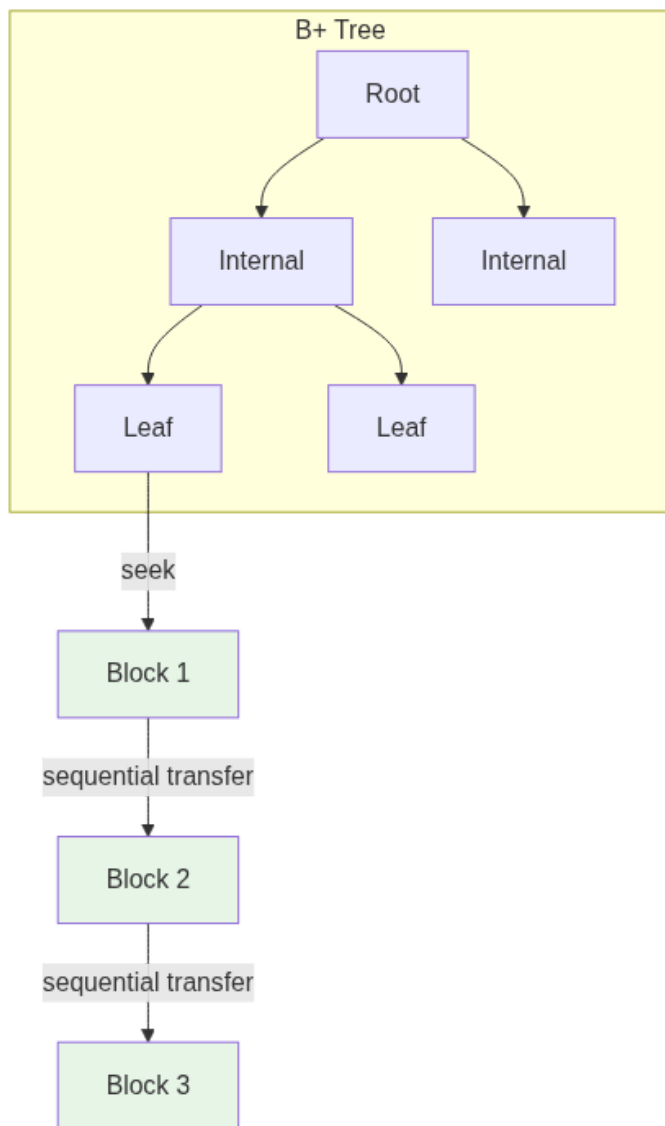
A3 — Primary index (clustering index), equality on nonkey.

Multiple records may match. Records will be stored on *consecutive blocks* (since the file is ordered on the search key).

Cost: $h_i \cdot (t_S + t_T) + t_S + b \cdot t_T$

Where b is the number of blocks containing the matching records.

The first part ($h_i \cdot (t_S + t_T)$) is the cost of traversing the index tree. The second part ($t_S + b \cdot t_T$) is *one seek to reach the first data block, followed by sequentially reading b blocks*.



A4 — Secondary index, equality on key.

Cost: $(h_i + 1) \cdot (t_S + t_T)$

Same as A2. If the search key is a candidate key, *at most one record will match*, so the cost is the same as for a clustering index on a key.

A5 — Secondary index, equality on nonkey.

Each matching record may reside on a *different block* (unlike the clustering case where matching records are contiguous).

$$\text{Cost: } (h_i + n) \cdot (t_S + t_T)$$

Where n is the **number of matching records**. Each matching record potentially requires a separate seek + transfer.

This cost can become very large because each record fetch requires an I/O operation. If the number of records to be fetched is large, it may actually be *cheaper to do a linear file scan* than to use the secondary index.

15.3.2 - Selections Involving Comparisons

Selections of the form $\sigma_{A \leq v}(r)$ or $\sigma_{A \geq v}(r)$ can be implemented using:

A6 — Primary index (clustering index), comparison.

For $A \geq v$: use the index to find the first tuple with $A = v$, then scan the file sequentially from there.

$$\text{Cost: } h_i \cdot (t_S + t_T) + t_S + b \cdot t_T$$

For $A \leq v$: *no need to consult the index*. Simply scan the file from the beginning until the first tuple with value greater than v , since the file is sorted.

$$\text{Cost: } t_S + b \cdot t_T$$

A7 — Secondary index, comparison.

For either direction ($\leq$ or $\geq$), the index must be used.

$$\text{Cost: } (h_i + n) \cdot (t_S + t_T)$$

In a secondary index, records are *not physically ordered* by the search key, so the shortcut of reading from the beginning doesn't apply. Each matching record must be fetched individually.

15.3.3 - Implementation of Complex Selections

Conjunction (AND): $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$

- **A8** — Use one index for the **most selective condition** (cheapest index), then *check the remaining conditions in memory*.
- **A9** — Use a **composite (multikey) index** that covers all conditions.
- **A10** — Intersection of identifiers: use *indices for each condition separately*, take the *intersection of the record pointers*, then fetch the matched records.

Disjunction (OR): $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$

If *all conditions have available indices*, use them and take the *union of the record pointers*. Otherwise, *a linear file scan is the only option* because we cannot skip any record that might satisfy one of the conditions.

15.5 - Join Operation

Several algorithms can be used to implement joins:

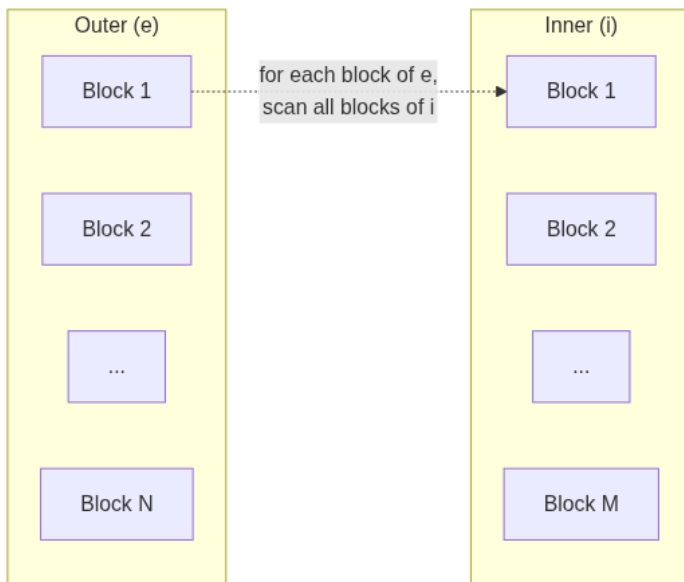
- Nested-loop join
- Block nested-loop join
- Indexed nested-loop join
- Merge join
- Hash join

We use the terms **outer relation** (*e*) and **inner relation** (*i*). The choice of which relation is outer and which is inner can make a *significant difference in the cost*.

15.5.2 - Block Nested-Loop Join (BNLJ)

A variant of the nested-loop join that operates on *blocks rather than individual tuples*.

```
for each block Be of e do
  for each block Bi of i do
    for each tuple te in Be and each tuple ti in Bi do
      test pair (te, ti) to see if they satisfy the join condition
```



Cost: $b_e \cdot (1 + b_i) \cdot (t_T + t_S)$

The number of blocks that must be scanned for the inner relation is *determined by the number of blocks of the outer*. Therefore, it is *better to use the smaller relation as the outer* (if $b_e < b_i$, total accesses are fewer).

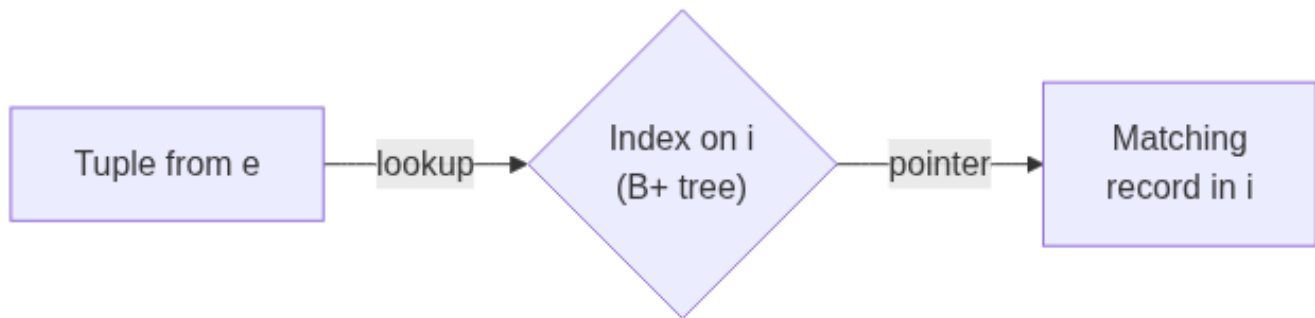
Worst case estimation: each block access is random, so the cost includes a seek + transfer for every single block read.

If we have B buffer blocks available in memory, we can read $B - 2$ *blocks of the outer relation at a time* (reserving one for the inner and one for the output), reducing the total number of scans of the inner relation.

15.5.3 - Indexed Nested-Loop Join (INLJ)

If an index is available on the *join attribute of the inner relation*, index lookups replace the file scan of the inner relation.

```
for each block  $B_e$  of  $e$  do
  for each tuple  $t_e$  in  $B_e$  do
    use the index on  $i$  to look up tuples matching  $t_e$ 
```



Cost: $b_e \cdot (t_T + t_S) + n_e \cdot c$

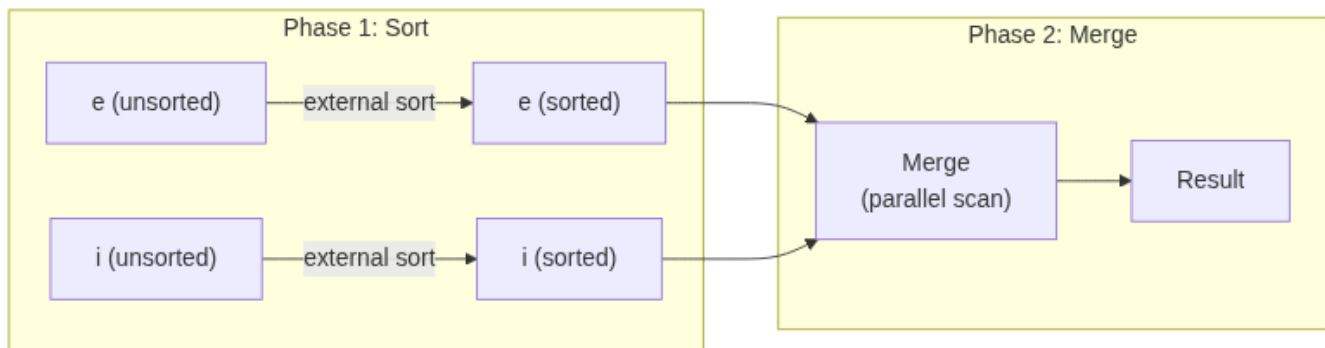
Where c is the cost of a *single selection on the inner relation using the index*. This is the cost formula from the selection algorithms:

- For a key attribute: $c = (h_i + 1) \cdot (t_S + t_T)$
- For a nonkey attribute: $c = (h_i + n) \cdot (t_S + t_T)$

The INLJ can be *much cheaper than BNLJ* when the index is highly selective (few matches per lookup).

15.5.4 - Sort-Merge Join (SMJ)

Can be used to compute **natural joins** and **equi-joins**. Both relations are *sorted on the join attribute*, then **merged** in a single pass (like merge in mergesort).



The sort phase uses **external sorting** when the relation does not fit in memory. An external sort of a relation with b blocks and M buffer blocks requires $\lceil \log_{M-1}(b) \rceil$ *passes*. Each pass reads and writes every block $\rightarrow 2 \times b$ I/O operations per pass.

Sort cost for one relation: $2 \cdot b_r \cdot \lceil \log_{M-1}(b_r) \rceil \cdot (t_T + t_S)$

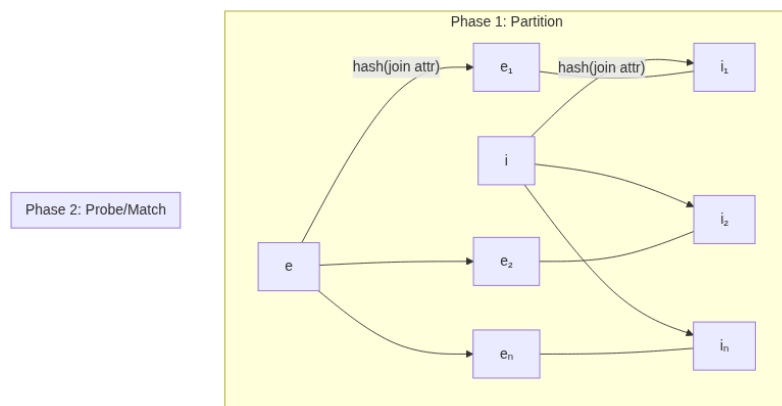
Total cost (sort + merge):

$$(b_e + b_i + 2 \cdot b_e \cdot \lceil \log_{M-1}(b_e) \rceil + 2 \cdot b_i \cdot \lceil \log_{M-1}(b_i) \rceil) \cdot (t_T + t_S)$$

The merge phase reads both sorted relations *sequentially once* $\rightarrow b_e + b_i$ accesses.

15.5.5 - Hash Join (HJ)

Also applicable for equi-joins and natural joins. Uses a **hash function** h on the join attribute to *partition both relations*.



Phase 1 — Partitioning: use h to split both relations into partitions. Records that *could possibly match must be in the same partition* (same hash value).

Partitioning cost: $2 \cdot (b_e + b_i) \cdot (t_T + t_S)$

(read all blocks + write all partitions)

Phase 2 — Probing: for each pair of partitions (e_k, i_k) with the same hash value, *load one into memory and probe with the other*. The join is done by *brute force in memory*, so no additional disk I/O.

Probing cost: $(b_e + b_i) \cdot (t_T + t_S)$

Total cost: $3 \cdot (b_e + b_i) \cdot (t_T + t_S)$

The hash join is efficient when *each partition fits in memory*. If a partition is too large, recursive partitioning is needed.

Optimización: Capítulo 16, de 16.1 a 16.4

Chapter 16 - Query Optimization

16.1 - Overview of Query Optimization

The task of the query optimizer is to generate an *efficient query-evaluation plan*.

A query expressed in SQL can typically be translated into *several equivalent relational-algebra expressions*. Moreover, for each relational-algebra expression, there are usually *multiple possible ways to evaluate it*, depending on the algorithms used for each operation.

The optimizer *does not aim to find the absolute best plan*. Finding the optimal plan may cost more than the query itself. Instead, the optimizer aims to *avoid the worst plans and find a reasonably efficient one*.

Spending 10 minutes to find the optimal plan that runs in 1 ms → total ≈ 10 min.
Spending 1 second to find a good-enough plan that runs in 1 sec → total ≈ 2 sec.

The optimizer relies on:

1. **Equivalence rules** — to generate alternative expressions.
2. **Heuristics** — to prune the search space.
3. **Cost estimation** — for each operation.
4. **Cardinality estimation** — for intermediate results.

16.2 - Transformation of Relational Expressions

16.2.1 - Equivalence Rules

Two relational-algebra expressions are said to be **equivalent** if they generate the *same set of tuples on every legal database instance*.

Selection:

- **Conjunctive decomposition:**

$$\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

- **Disjunction as union:**

$$\sigma_{\theta_1 \vee \theta_2}(E) \equiv \sigma_{\theta_1}(E) \cup \sigma_{\theta_2}(E)$$

- **Commutativity:**

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

Set operations:

- Union, intersection, and join are **commutative**:

$$E_1 \cup E_2 \equiv E_2 \cup E_1$$

- And **associative**:

$$(E_1 \cup E_2) \cup E_3 \equiv E_1 \cup (E_2 \cup E_3)$$

Join:

- Theta-join is **associative**:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 \equiv E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

This allows changing the *order in which joins are performed*, which is critical for optimization since join order can drastically affect cost.

16.3 - Estimating Statistics of Expression Results

The **catalog** stores the following statistical information for each relation r :

Statistic	Meaning
n_r	Number of tuples in r
b_r	Number of blocks containing tuples of r
l_r	Size of a tuple of r (in bytes)
f_r	Blocking factor: number of tuples per block
$V(A, r)$	Number of distinct values of attribute A in r

For indices:

Statistic	Meaning
-----	-----
$Height_{I(A,r)}$	Height of the B+ tree index I on attribute A of r
$Length_{I(A,r)}$	Number of leaf-level blocks in index I

Estimation of the Size of Selections

Assuming **uniform distribution** of values:

Equality: $\sigma_{A=a}(r) \rightarrow$ estimated size: $\frac{n_r}{V(A, r)}$

Range: $\sigma_{A \leq v}(r)$:

- If $v < \min(A, r)$: $\rightarrow 0$
- If $v > \max(A, r)$: $\rightarrow n_r$
- Otherwise: $\rightarrow n_r \cdot \frac{v - \min(A, r)}{\max(A, r) - \min(A, r)}$

Conjunction $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$: assuming **independence** between conditions, if s_i is the estimated number of tuples satisfying θ_i :

$$n_r \times \frac{s_1 \times s_2 \times \dots \times s_n}{n_r^n}$$

Disjunction $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$:

$$n_r \times \left(1 - \prod_{i=1}^n \left(1 - \frac{s_i}{n_r} \right) \right)$$

The probability that a tuple satisfies at least one condition == 1 – probability of satisfying none.

Estimation of the Size of Joins

Cartesian product $r \times s$: $n_r \cdot n_s$

If the join condition involves a key of s : at most n_r tuples in the result (each tuple in r matches at most one in s).

If the join condition involves a key of r : at most n_s .

General case (no key involved):

$$\min \left(\frac{n_r \cdot n_s}{V(A, s)}, \frac{n_r \cdot n_s}{V(A, r)} \right)$$

For each tuple of r , there are on average $n_s/V(A, s)$ matching tuples in s . The *minimum of the two perspectives* is taken as the estimate.

Other Operations

Operation	Estimated size
$\pi_A(r)$ (with DISTINCT)	$V(A, r)$
$G\gamma_A(r)$ (aggregation)	$V(G, r)$
$r \cup s$	$n_r + n_s$
$r \cap s$	$\min(n_r, n_s)$
$r \bowtie s$ (left outer join)	$\ r \bowtie s\ + n_r$
$r \Join s$ (full outer join)	$\ r \bowtie s\ + n_r + n_s$

16.4 - Choice of Evaluation Plans

16.4.1 - Cost-Based Optimization

The optimizer generates alternative evaluation plans using equivalence rules, estimates the cost of each plan, and *chooses the plan with the lowest estimated cost*.

16.4.2 - Heuristic Optimization

Heuristic rules reduce the search space without computing costs:

1. **Perform selections as early as possible:** pushing selections down the tree reduces the number of tuples that subsequent operations must process.
2. **Perform projections early:** discarding unnecessary columns reduces the size of intermediate relations.
3. **Perform the most restrictive joins first:** starting with joins that produce fewer results reduces the size of intermediate relations.

These heuristics *do not guarantee the optimal plan*, but they *eliminate many clearly bad options*. Real optimizers combine heuristic-based decisions with cost-based decisions.

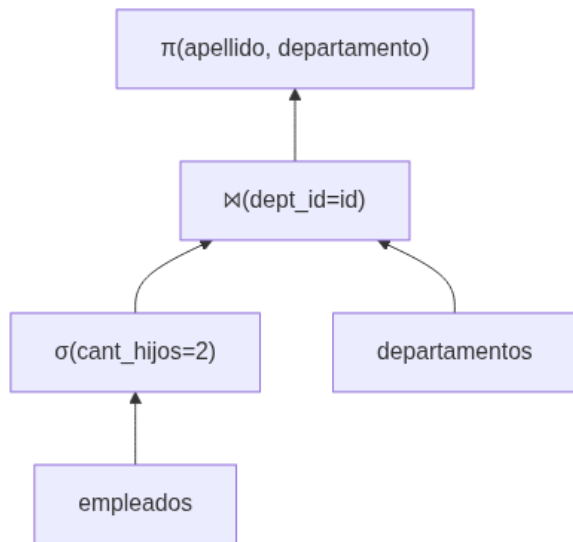
Evaluation of Expressions

The algebra expression is represented as a **tree of operators**:

- **Leaves** → base relations (tables).
- **Internal nodes** → operators (σ , π , $\bowtie$, etc.).

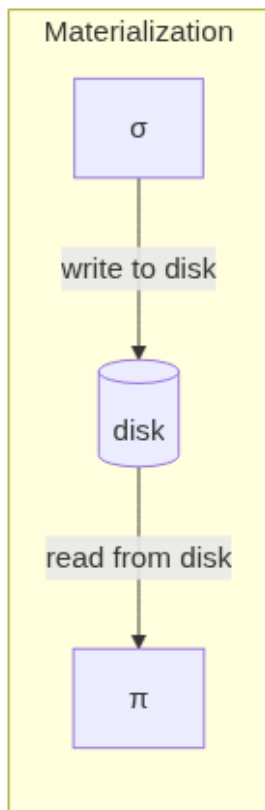
- **Root** → final operator producing the result.

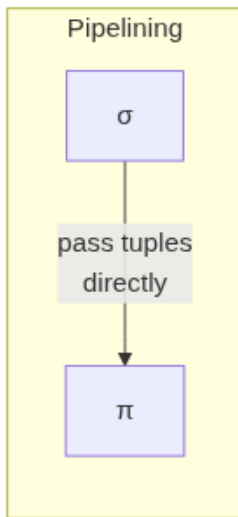
The tree is evaluated **bottom-up** (from leaves to root). Each node produces an intermediate relation that feeds into its parent.



Materialization: intermediate results are *written to disk*. The next operator reads them back. Cost: $\lceil n_r / f_r \rceil$ blocks to write, plus reading them back.

Pipelining: operators are *chained* so that results flow from one to the next *without writing to disk*.





Advantages of pipelining:

- *Reduces the number of temporary files* written to disk.
- *Reduces latency*: first results can appear before the entire operation completes.

Pipelining works when the operator above can start with *partial results*. It *does not work* when the operator needs all data first (e.g., sorting).

A selection followed by a projection: as σ emits records, π processes them immediately → *pipelining works*.

A sort needs all records before it can produce output → *pipelining does not work*.